Lab Manual 11 SEMAPHORES

84 WHAT IS A SEMAPHORE ?

1. Dijkestra proposed a significant technique for managing concurrent processes for complex mutual exclusion problems. He introduced a new synchronization tool called Semaphore.

2. Posix Semaphores are Inter Process or Threads synchronization technique, just like mutex.

3. A semaphore is an integer value variable, which can be incremented and unlocked or decremented and locked by threads or processes.

85 POSIX TYPES OF SEMAPHORE

POSIX Semaphores are of 2 types:

1. Named Semaphore
2. Memory-mapped Semaphore

86 UNNAMED SEMAPHORES (MEMORY-BASED SEMAPHORES)

1. No name is associated with these semaphores
2. Provides synchronization between threads and between related processes
3. Placed in a region of main memory that is shared between processes/threads
4. For threads this is done by simply making the semaphore a global variable

87 KINDS OF SEMAPHORES

Depending upon the value a semaphore is made to hold, it can be:

1. Binary semaphore
2. Counting semaphore

88 USES OF SEMAPHORES

Semaphores can be use for synchronization between Threads/Processes. It also provide a way to avoid Dead-locks.

1. **For placing Locks**

   Just like mutex (can me binary or counting).

(a) Counting Semaphores:

Permit a limited number of threads to execute a section of the code.

(b) Binary Semaphores:

Permit only one thread to execute a section of the code.

2. **Semaphores As Condition Variables**

(a) Semaphores are also useful when a thread wants to halt its progress waiting for a condition to become true.

(b) Communicate information about the state of shared data.

89 SEMAPHORES VS MUTEX

1. Mutex can be locked or unlocked, like binary semaphore. Semaphores (counting) can have multiple values.

2. A locked mutex can be unlocked by the thread holding the lock. A locked semaphore can be unlocked by any thread.

3. Semaphore has state (value of semaphore) associated with it.

4. Mutex and condition variables are used together in most scenario. Looking at their functionality, it can be thought as : Semaphore = Mutex + Condition Variable

5. Posix Named Semaphore are kernel persistent. Posix Memory based semaphore, Posix Condition Variable and Posix Mutex are process persistent.

100 SEMAPHORE SYSTEM CALLS

```
# include      <semaphore . h>
```

```
int      sem_init ( ) ;
```

```
int      sem_wait ( ) ;
```

```
int      sem_trywait ( ) ;
```

```
int      sem_post ( ) ;
```

```
int     sem_destroy ( ) ;
```

101 CREATE A SEMAPHORE

```
int     sem_init ( sem_t *sem ,     int     pshared , unsigned      int value )
```

1. **sem:** Target semaphore
2. **pshared:** The pshared argument indicates whether this semaphore is to be sharedbetween the threads of a process, or between processes.
   (a) **0:** only threads of the creating process can use the semaphore.
   (b) **Non-0:** other processes can use the semaphore.
3. **value:** Initial value of the semaphore.

EXAMPLE:

```
# include <semaphore . h> sem_t s ;
sem_init(&s ,           0,  1)
```

We declare a semaphore s and initialize it to the value 1 by passing 1 in as the third argument. The second argument to sem init() will be set to 0 in all of the examples we'll see; this indicates t hat the semaphore is shared between threads in the same process.

102 SEMAPHORE OPERATIONS

1. **sem_wait()** decrements (locks) the semaphore pointed to by sem.

```
int     sem_wait ( sem_t *sem)
```

   (a) If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns(gets lock), immediately.
   (b) If the value of the semaphore is negative, the calling thread blocks; one of the blocked threads wakes up when another thread calls sem_post()

2. **sem_post()** does not wait for some particular condition to hold like sem_wait() does.

```
int     sem_post ( sem_t *sem)
```

Rather, it simply increments the value of the semaphore and then, if there is a thread waiting to be woken, wakes one of them up.

```
int  sem_wait ( sem_t*s )       {
                        decrement       the  value      of      semaphore       s       by one
                                wait if value       of      semaphore       s is    negative




}
int  sem_post ( sem_t*s )
                        increment       the  value      of      semaphore       s       by one



                if there      are     one or more     threads     waiting ,     wake one
}
```

3. **sem_trywait()** is the version of of the sem_wait() which does not block.

```
int     sem_trywait ( sem_t * sem)
```

Decreases the semaphore by one if the semaphore does not equal to zero. If it is zero it does not block, returns zero with error code EAGAIN.

4. **sem_destroy** releases the resources that semaphore has and destroys it

```
int     sem_destroy ( sem_t * sem)
```

103 EXAMPLES

EXAMPLE 01: SEMAPHORE A BINARY LOCK

```
# include <s t d i o . h>
# include < s t d l i b . h>
# include <signal . h>
# include <pthread . h>
# include        <semaphore . h>
# include         <unistd . h>

// Using        semaphore  as  a  Binary                  lock for IPC/Syncronization

sem_t sem ; void * f1 ( void *arg ) { sem_wait(&sem )
;
                printf ( " Thread with ID:%ld got lock\n" , pthread_self ( ) ) ; sleep ( 2 ) ;
                sem_post(&sem ) ;
                pthread_exit (NULL) ;


}

int


        main ( ) { pthread_t tid1 , tid2 ; sem_init(&sem , 0 , 1 ) ;
                pthread_create(&tid1      ,NULL,     f1      ,NULL)     ;
                pthread_create(&tid2      ,NULL,     f1      ,NULL)     ;
}pthread_exit (NULL) ;
```

EXAMPLE 02: SEMAPHORE A COUNTING VARIABLE

```
# include <s t d i o . h>
# include < s t d l i b . h>
# include <signal . h>
# include <pthread . h>
#  include <semaphore . h> #
include <unistd . h> sem_t sem ;
void * f1 ( void *arg )
{
        int     i =1 , value ;
        sem_wait(&sem ) ; sem_getvalue(&sem ,
        &value ) ;
          printf ( " Thread     with  ID %ld        return  from    wait ( ) and
                                          Sem     state   is              :%d\n" , pthread_self ( ) , value ) ;
        pthread_exit ( 0 ) ;
}
int  main ( )
{
         pthread_t      tid [ 5 ] ;
        int     value , i ;
        sem_init(&sem , 0 , 3 ) ; for ( i =0; i <5; i ++){ pthread_create(& tid
        [ i ] ,NULL, f1 ,NULL) ;
        }

        sleep ( 3 ) ; printf ( "Main : Going to Post\n" )
        ;
        sem_post(&sem ) ; sem_getvalue(&sem ,
        &value ) ;
        printf ( "Main : After Post sem state :%d \n" , value ) ; sleep ( 3 ) ; printf
        ( "Main : Going to Post\n" ) ;
        sem_post(&sem ) ; sem_getvalue(&sem ,
        &value ) ;
        printf ( "Main : After Post sem state :%d \n" , value ) ; pthread_exit ( 0 ) ;

}
```

EXAMPLE 03: SEMAPHORE A CONDITIONAL VARIABLE

```
# include <s t d i o . h>
```

```
# include < s t d l i b . h>                                    for    IPC/Syncronization
# include <signal . h>
# include <pthread . h>
 # include      <semaphore . h>
 # include       <unistd . h>

// Using        semaphore    as  a  condtion          variable

sem_t sem ; void * f1 ( void *arg ) { printf ( " Child
: begin\n" ) ;                                                                    sem_post(&sem ) ;
                                          printf ( " Child : end\n" ) ; pthread_exit (NULL)
              ;
```

```
}
```

```
int
      main ( ) { pthread_t tid ; sem_init(&sem , 0 , 0 ) ; printf
            ( " parent :    begin\n" ) ; pthread_create(&tid
            ,NULL, f1 ,NULL) ;
            sem_wait(&sem ) ;
             printf ( " parent : end\n" ) ; pthread_exit (NULL) ;


}
```